

Qwen3-Coder-Next Tier 3

Part 6 Detailed Implementation Specification

Testing & Review

Engineering specification for the testing and review gate layer

Document intent

This part defines the quality gates that sit between code generation and broader autonomy. It specifies how the system executes tests, interprets failures, performs code review, and turns both signals into a deterministic pass/reject decision.

1. Purpose

This part exists to stop the agent from treating compiled code or a clean diff as proof of success. Testing proves that the change behaves as intended. Review proves that the change is acceptable to ship. The two signals are related but not interchangeable, so this part keeps them separate and then merges them into a gate that later stages can trust.

The problem it solves is predictable: code generation alone can produce broken, incomplete, unsafe, or subtly wrong output. A test-only system can still ship poor engineering decisions. A review-only system can miss runtime failures. This part creates the first strong quality boundary in the system.

Future parts depend on this layer because the Evaluator, Failure Recovery, and Deployment gates all need a clean, structured record of what failed, why it failed, and whether the fix should be retried, rejected, or escalated.

Key rule

Testing answers: does it work? Review answers: is it acceptable? The gate answers: should this move forward now?

2. Scope

Included in this part:

- test execution orchestration for the current worktree
- test suite selection and command resolution
- raw output capture and result normalization
- code review rubric, finding extraction, and severity assignment
- combined gate decision generation
- artifact persistence for test and review evidence
- feedback packaging for the Coding Agent
- failure categorization for downstream recovery logic

Excluded from this part:

- semantic task alignment checks handled by the Evaluator
- knowledge-graph navigation and repository intelligence
- long-term memory retrieval and persistence policy

- deployment execution and human approval workflow
- recovery strategy selection beyond gate failure reporting

3. System Overview

The high-level flow is simple on purpose. Simplicity is rare and therefore suspicious, but it is correct here.

```
User
|
LangGraph Orchestrator
|
Coding Agent -> Worktree
|
Test Orchestrator -> Test Report
|
Review Orchestrator -> Review Report
|
Gate Coordinator -> Pass / Reject
|
If reject: feedback to Coding Agent
|
If pass: handoff to next stage
```

The test path executes code in the isolated worktree, collects machine-readable output, and normalizes it into a stable report. The review path inspects the diff and surrounding context, applies the rubric, and emits findings that can be acted on. The gate coordinator combines both reports and decides whether the candidate is allowed to continue.

Pseudoflow:

1. Receive candidate change and worktree reference
2. Select test scope
3. Execute tests and capture report
4. Run review against diff and rubric
5. Normalize both outputs
6. Merge into a gate decision
7. Persist artifacts and return feedback

4. Internal Architecture

This part has two parallel analysis paths and one decision path. The parallel paths keep concerns separate so that tests do not become reviews and reviews do not become hidden tests.

Component	Responsibility	Primary Output
Test Orchestrator	Launches the chosen test command in the current worktree and captures raw runtime evidence.	Raw execution log + status

Result Normalizer	Converts diverse test output formats into a single report schema.	Structured TestReport
Review Orchestrator	Feeds the change diff and context into a deterministic review rubric.	Structured ReviewReport
Gate Coordinator	Combines test and review results into pass/reject output.	GateDecision
Artifact Store	Persists reports, logs, and review comments for later agents.	Versioned artifacts
Feedback Adapter	Formats the rejection payload for the Coding Agent.	Actionable feedback bundle

Dependencies are intentionally one-way:

```

Coding Agent
|
Test Orchestrator --> Result Normalizer
|
Review Orchestrator
|
Gate Coordinator
|
Artifact Store + Feedback Adapter

```

Required abstractions include a test adapter interface, a review rubric interface, a report serializer, and a gate policy interface. These abstractions let the implementation swap test runners or review strategies later without changing the gate contract.

Architectural constraint

The gate coordinator must not silently reinterpret a failure as a pass. It can only classify, combine, and forward evidence.

5. Project Structure

Recommended directory structure:

```

quality/
  testing/
    orchestrator/
    adapters/
    parsers/
    fixtures/
  review/
    rubric/
    analyzers/
    findings/
  gates/

```

```
coordinator/  
policies/  
decisions/  
schemas/  
artifacts/  
config/  
logs/  
docs/
```

Folder intent:

- `quality/testing/orchestrator` - Coordinates suite selection, command execution, and retry-safe test launches.
- `quality/testing/adapters` - Contains adapters for pytest, npm, cargo, and other runners.
- `quality/testing/parsers` - Normalizes raw runner output into structured reports.
- `quality/testing/fixtures` - Stores golden outputs, failure samples, and reproducibility cases.
- `quality/review/rubric` - Defines the review criteria used to score and classify changes.
- `quality/review/analyzers` - Implements diff inspection, policy checks, and issue extraction.
- `quality/review/findings` - Defines the canonical structure for review findings.
- `quality/gates/coordinator` - Merges test and review signals into final decisions.
- `quality/gates/policies` - Stores thresholds and escalation rules.
- `quality/schemas` - Shared data contracts used by every component.
- `quality/artifacts` - Persisted reports, logs, and evidence bundles.
- `quality/config` - Runtime configuration for suite selection, thresholds, and review settings.
- `quality/logs` - Operational logs and diagnostic traces.

6. Data Contracts

The system should communicate using stable report objects, not ad hoc strings. Humans inventing ad hoc text formats is how everyone ends up debugging a pile of lowercase regret.

Contract	Required Fields	Notes
TestInvocation	task_id, worktree_id, runner, command, timeout, cwd, env_profile	Describes one execution request.
TestReport	task_id, worktree_id, suite_name, exit_code, status, summary,	Normalized test result.

	failed_cases, logs_ref, artifacts_ref	
ReviewInstruction	task_id, diff_ref, rubric_id, context_budget, policy_profile	Tells the review engine what to inspect.
ReviewFinding	id, severity, category, summary, evidence_ref, recommendation	Atomic review output.
ReviewReport	task_id, worktree_id, score, findings, overall_status, reviewer_notes, artifacts_ref	Structured review result.
GateDecision	task_id, decision, reasons, blocking_findings, retryable_findings, next_action	Final pass/reject signal.
FeedbackBundle	task_id, user_visible_summary, machine_actionable_notes, rerun_hints	Returned to the Coding Agent.

Example structure for a normalized test report:

```
TestReport
- task_id: feature-017
- suite_name: unit
- status: fail
- exit_code: 1
- summary: 3 tests failed, 41 passed
- failed_cases:
  - name: test_parser_handles_empty_input
    location: tests/parser/test_empty.py:44
    failure_class: assertion
- logs_ref: artifacts/test-runs/feature-017/unit.log
- artifacts_ref: artifacts/test-runs/feature-017/report.json
```

Example structure for a review finding:

```
ReviewFinding
- severity: high
- category: error-handling
- summary: Missing fallback path for invalid configuration
- evidence_ref: diff://src/config/loader.py
- recommendation: Add explicit validation before parsing user input
```

7. State Management

The state for this part is mostly per-task and per-worktree. That keeps quality checks repeatable and avoids stale decisions leaking between unrelated runs.

State Type	Owner	Persistence	Consumption
Test run state	Test Orchestrator	Artifact store + task state	Used by gate coordinator and recovery

Review state	Review Orchestrator	Artifact store + task state	Used by gate coordinator and evaluator
Gate state	Gate Coordinator	Task state + logs	Used by LangGraph routing
Policy state	Config layer	Persistent config	Used by every run

State should change only at clear boundaries: invocation created, test started, test finished, review started, review finished, gate decided. Intermediate mutation must remain internal so the external state never becomes ambiguous.

Future agents consume state as a compact evidence bundle: the current candidate, the test report, the review report, and the gate decision. They should not need to reconstruct history from raw logs unless debugging a failure.

Persistence rule

Artifacts are immutable once written. New attempts create new artifact versions instead of overwriting prior evidence.

8. Component Specifications

Test Orchestrator

Inputs: task candidate, worktree reference, test policy

Outputs: raw runner output, command status, execution metadata

Responsibilities: selects and launches tests, enforces timeout and isolation

Failure conditions: runner unavailable, timeout, non-zero exit

Future integration points: feeds evaluator and recovery layers

Result Normalizer

Inputs: raw output, log path, suite metadata

Outputs: canonical TestReport

Responsibilities: parses runner-specific formats into one schema

Failure conditions: parse failure, missing metadata

Future integration points: supports benchmark harness later

Review Orchestrator

Inputs: diff, review rubric, context summary

Outputs: ReviewReport with findings

Responsibilities: evaluates code quality, structure, and risk

Failure conditions: insufficient context, rubric mismatch

Future integration points: feeds evaluator and human approval

Gate Coordinator

Inputs: TestReport + ReviewReport

Outputs: GateDecision + FeedbackBundle

Responsibilities: decides pass/reject and composes feedback

Failure conditions: inconsistent reports, missing artifacts

Future integration points: handoff to recovery and deployment

Artifact Publisher

Inputs: reports and logs

Outputs: artifact references

Responsibilities: stores evidence and makes it retrievable

Failure conditions: write failure, path collision

Future integration points: shared by later parts

9. Implementation Sequence

Step 1 - Define schemas and artifact layout

Do this first so every later component speaks the same language. Output: shared contracts, directories, and naming rules.

Why it comes now: No functional dependency. This is the foundation for everything else.

Step 2 - Build test adapter interface

Create the abstraction that can launch runners consistently. Output: a common invocation wrapper.

Why it comes now: Requires schema definitions only.

Step 3 - Add runner-specific adapters

Implement pytest, npm, and any first-supported runner. Output: real execution support.

Why it comes now: Depends on the adapter interface.

Step 4 - Normalize raw results

Turn raw process output into TestReport records. Output: stable machine-readable reports.

Why it comes now: Depends on step 3.

Step 5 - Implement review rubric and analyzers

Create deterministic review rules and finding extraction. Output: ReviewReport records.

Why it comes now: Depends on diff access and schema definitions.

Step 6 - Build gate coordinator

Merge test and review results into one decision path. Output: GateDecision and FeedbackBundle.

Why it comes now: Depends on both reports.

Step 7 - Wire artifact publishing

Persist every report and attach references to the task state. Output: immutable evidence bundles.

Why it comes now: Depends on report objects.

Step 8 - Integrate with LangGraph

Connect pass/reject outcomes to the upstream workflow. Output: real routing behavior.

Why it comes now: Depends on gate coordinator.

Step 9 - Add fixtures and failure injection

Prove the gate behaves correctly under broken commands, parse errors, and bad code. Output: regression suite.

Why it comes now: Depends on the full stack.

10. Validation Strategy

Validation must cover both correctness and stability. A gate that fails randomly is not a gate. It is a superstition with file I/O.

- unit tests for adapters, parsers, rubric logic, and decision thresholds
- integration tests using a small synthetic repository with passing and failing scenarios
- golden-file fixtures for runner outputs and review findings
- failure injection for timeouts, malformed logs, missing files, and non-zero exits
- deterministic comparison of emitted artifacts across reruns
- regression checks for review severity classification and gate decision consistency

11. Deliverables

- shared schema package for test and review reports
- test execution wrapper with at least one runner adapter
- result parser and normalized report generator
- review rubric engine and finding extractor
- gate coordinator and feedback bundle builder
- artifact storage layout and retention rules
- fixtures for success, failure, and parser edge cases
- operator-facing configuration file
- developer documentation for running the gate locally

12. Acceptance Criteria

Check	Verification Method
A test command can be executed in an isolated worktree and its exit status is captured correctly.	Run an integration test against a fixture repository.
A failing test suite produces a normalized TestReport with failed case details.	Compare output JSON against a golden file.
A code diff can be reviewed and converted into a ReviewReport with severity labels.	Validate rubric output on curated review fixtures.
The gate coordinator can reject a change because of test failure, review failure, or both.	Exercise three negative scenarios and assert decisions.
Artifacts are written to stable paths and are retrievable after the run completes.	Check the artifact manifest and file existence.
The feedback bundle contains actionable, machine-readable reasons for rejection.	Inspect the task output payload.
A rerun with the same input produces the same decision and equivalent normalized artifacts.	Run determinism regression tests.

A part is complete only when every acceptance criterion can be demonstrated in a clean run, from scratch, without manual patching of outputs. That last part is the important bit, because humans adore hand-waving past broken automation.

13. Common Failure Modes

- treating raw runner output as the final data model

- letting review comments drift into subjective prose instead of actionable findings
- missing timeouts, which turns the gate into a denial-of-service against itself
- mixing gate policy with runner implementation
- overwriting evidence instead of versioning it
- marking a change as acceptable because one signal passed while the other failed
- building review logic that cannot be tuned without code changes

14. Future Integration

This part feeds the next stages directly.

Next Stage	How Part 6 Connects
Part 7 - Evaluation Layer	Provides structured reports that the Evaluator uses to detect wrong-problem solutions.
Part 8 - Failure Recovery	Supplies precise failure classes and remediation hints.
Part 9 - Repository Intelligence	Uses review artifacts to understand recurring code structure issues.
Part 10 - Code Knowledge Graph	Consumes review and test evidence to trace impact of risky changes.
Part 14 - Deployment & Human Approval	Supplies the quality gate records that justify release decisions.

The key handoff is semantic: later parts should not need to guess whether a task failed because the code was wrong, the test was wrong, or the review was incomplete. Part 6 must make that distinction explicit.

15. Completion Checklist

- Schema package exists and is imported by both testing and review modules
- At least one real test runner adapter works end to end
- Raw test output is normalized into a stable TestReport
- Review rubric emits structured findings with severity and category
- Gate coordinator returns deterministic pass/reject decisions
- Evidence artifacts are persisted and retrievable
- Integration tests cover pass, fail, and ambiguous scenarios

- Regression fixtures lock down output shape and decision behavior
- Documented configuration is sufficient to run the part locally